GLOBETREK

INTELLIGENT TRAVEL COMPANION

Official Full-Stack Web Application Project Proposal

Project Type	Technology	Target Domain
Academic / Portfolio Full-Stack Web Application	React + Node.js + PostgreSQL	Travel & Tourism Technology

Prepared for:

Student Full-Stack Development Project

Sri Lanka

1. Executive Summary

GlobeTrek is a proposed full-stack web application designed to help travelers plan, organize, and manage trips from a single platform. The application focuses on practical travel-planning problems such as scattered trip information, difficulty managing budgets, organizing places to visit, and maintaining a clear itinerary.

The project will be developed as a learning-oriented but production-style application using modern web technologies. The first release will concentrate on a manageable Minimum Viable Product (MVP), while the architecture will leave room for future services such as maps, weather, AI-based recommendations, notifications, and travel-community features.

2. Problem Statement

Travel planning often requires users to combine information from multiple services: notes for itineraries, map applications for locations, spreadsheets for budgets, and separate websites for attractions and travel information. This fragmented workflow can make trip planning difficult to organize and maintain.

For a beginner full-stack developer, there is also a learning opportunity: the travel domain provides a realistic problem in which authentication, CRUD operations, relational data, APIs, responsive interfaces, and external integrations can be combined into one coherent system.

3. Proposed Solution

GlobeTrek will provide a centralized travel workspace where a user can create trips, define destinations and dates, organize planned activities, record travel expenses, and view trip information in a structured dashboard. The MVP will prioritize simplicity and reliability rather than attempting to implement every advanced feature at the beginning.

4. Objectives

- Build a complete full-stack web application from frontend to database.
- Provide secure user registration and authentication.
- Allow users to create, edit, view, and delete trips.
- Provide structured itinerary and place management for each trip.
- Provide a basic travel budget and expense tracker.
- Design a responsive and user-friendly interface for desktop and mobile.
- Create a backend REST API with clear separation between presentation, business logic, and data storage.
- Prepare the application for future integration with mapping, weather, and AI services.

5. Scope of the MVP

Module	Core Functions	Priority	Outcome
Authentication	Register, login, logout, protected routes	High	Users can securely access personal data
Trip Management	Create, edit, view, delete trips	High	Central trip workspace
Itinerary	Add dates, activities and notes	High	Organized day-by-day plan
Places	Save locations and categories	Medium	Personal place collection
Budget	Add expenses and calculate totals	Medium	Basic trip cost visibility
Dashboard	Summary of trips, dates and spending	Medium	Single overview screen

6. Future Enhancements

- Interactive maps and route visualization using a mapping service.
- Live weather information for destinations.
- AI-assisted destination and activity recommendations based on user preferences.
- Collaborative group trips and shared expenses.
- Travel journal with photos and memories.
- Notifications for itinerary events and planned activities.
- Local experiences and community-generated recommendations.

7. Target Users

Primary users are independent travelers, students, backpackers, couples, and small groups who want a simple way to organize trip information. The initial implementation will focus on individual users, with collaborative features planned for later versions.

8. Functional Requirements

ID	Requirement	Description
FR-01	User Registration	The system shall allow a new user to create an account.
FR-02	User Login	The system shall authenticate users and maintain a user session.
FR-03	Create Trip	An authenticated user shall be able to create a trip with a title, destination, start date and end date.
FR-04	Manage Trip	An authenticated user shall be able to view, edit and delete their own trips.
FR-05	Itinerary Item	A user shall be able to add activities or notes to a trip.
FR-06	Expense	A user shall be able to record an expense with amount, category and description.
FR-07	Trip Summary	The system shall calculate and display total recorded expenses for a trip.

FR-08	Data Isolation	Users shall only access records associated with their own account.
FR-09	Responsive UI	The application shall support common desktop and mobile screen sizes.

9. Non-Functional Requirements

- Security: Passwords must not be stored in plain text; authentication and authorization must be enforced on protected operations.
- Usability: Common actions should be easy to discover and require minimal steps.
- Performance: Typical dashboard and CRUD operations should provide responsive feedback under normal project-scale usage.
- Maintainability: The codebase should use modular components, clear naming, reusable services, and environment variables for configuration.
- Scalability: The database and API design should allow additional travel features to be introduced without redesigning the entire system.

10. Proposed Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Component-based user interface and development tooling
Styling	CSS / Tailwind CSS	Responsive visual design and reusable styles
Backend	Node.js + Express	REST API and application logic
Database	PostgreSQL	Relational storage for users, trips, itinerary items and expenses
ORM	Prisma (planned)	Database schema, queries and migrations
Version Control	Git + GitHub	Source control and collaboration
API Testing	Postman	Testing REST endpoints
Deployment	Vercel + backend hosting	Public deployment of frontend and server components

11. High-Level System Architecture

The proposed architecture separates the application into three primary layers: a React frontend, a Node.js/Express backend API, and a PostgreSQL database. External services such as maps, weather, and AI can be connected to the backend later without coupling them directly to the frontend.

- React Frontend -> HTTP/JSON -> Express REST API
- Express API -> ORM / SQL -> PostgreSQL Database
- Express API -> External Travel APIs (future)
- Authentication -> Protected API Routes -> User-specific Data

12. Initial Database Design

- Users: id, name, email, password hash, created_at
- Trips: id, user_id, title, destination, start_date, end_date, description
- ItineraryItems: id, trip_id, date, title, location, notes
- Expenses: id, trip_id, category, amount, description, expense_date
- Places (optional MVP extension): id, trip_id, name, category, latitude, longitude, notes

13. Development Methodology

The project will follow an incremental development approach. Each stage will produce a working feature before the next stage begins. This reduces complexity and is appropriate for a beginner who is learning full-stack development while building the application.

Phase	Focus	Main Deliverables
Phase 1	Project setup & UI	React, Vite, routing, responsive layout, reusable components
Phase 2	Authentication	User registration/login and protected routes
Phase 3	Backend foundation	Express server, API routes, validation and error handling
Phase 4	Database	PostgreSQL schema, Prisma models, migrations and CRUD operations
Phase 5	MVP integration	Connect frontend to backend and implement trip/itinerary/expense flows
Phase 6	Testing & deployment	API tests, UI testing, bug fixing, GitHub workflow and deployment
Phase 7	Advanced features	Maps, weather, AI and collaborative travel features

14. Risk Management

- Scope expansion: Advanced features will be deferred until the MVP is stable.
- Limited backend knowledge: Development will proceed module-by-module with API and database concepts learned alongside implementation.
- External API dependency: Third-party integrations will be isolated behind backend services so the core app remains usable if an API is unavailable.
- Security issues: Authentication, input validation, access control, environment variables and safe secret handling will be treated as core requirements.

15. Expected Outcomes

- A functional full-stack travel-planning web application.
- A documented REST API and relational database schema.
- Practical experience with React, Node.js, Express, PostgreSQL and Git.
- A portfolio project demonstrating frontend, backend, database and deployment skills.
- A foundation for future work in travel technology and AI-powered applications.

16. Conclusion

GlobeTrek is proposed as a practical, scalable learning project that addresses a recognizable travel-planning problem while providing a structured path from beginner frontend development into complete full-stack engineering. By building the MVP first and adding integrations progressively, the project can remain achievable while still providing a strong technical foundation for future travel-technology development.

17. Proposal Approval / Sign-Off

Project / Proposal Name	GlobeTrek - Intelligent Travel Companion
Prepared By	_____
Supervisor / Reviewer	_____
Date	_____